

C language Fundamentals

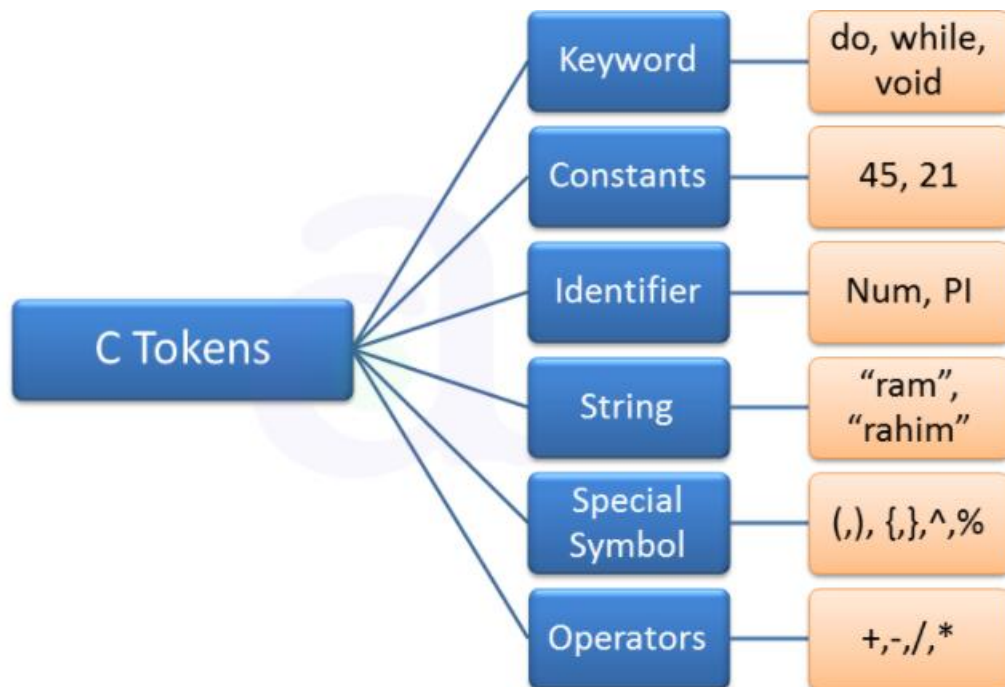
C Tokens

In C programs, each word and punctuation is referred to as a token. C Tokens are the smallest building block or smallest unit of a C program.

The compiler breaks a program into the smallest possible units and proceeds to the various stages of the compilation, which is called token.

C Supports Six Types of Tokens:

1. [Identifiers](#)
2. [Keywords](#)
3. [Constants](#)
4. [Strings](#)
5. [Operators](#)
6. Special Symbols



C Identifiers

Identifiers are names given to different entities such as constants, variables, structures, functions, etc.

Example:

```
int amount;  
  
double totalbalance;
```

In the above example, **amount** and **totalbalance** are identifiers.

int and **double** are keywords.

Rules for Naming Identifiers

- An identifier can only have alphanumeric characters (a-z , A-Z , 0-9) (i.e. letters & digits) and underscore(_) symbol.
- Identifier names must be unique
- The first character must be an alphabet or underscore.
- You cannot use a keyword as identifiers.
- Only the first thirty-one (31) characters are significant.
- It must not contain white spaces.
- Identifiers are case-sensitive.

C Keywords

The C Keywords must be in your information because you can not use them as a variable name.

You can't use a keyword as an identifier in your C programs, its reserved words in C library and used to perform an internal operation. The meaning and working of these keywords are already known to the compiler.

Outline:

[#_C Keywords List](#)

[#_Example Where and How Keywords are Used in the Program](#)

[#_Keywords in C - Video Tutorial](#)

C Keywords List

A list of **32 reserved keywords** in c language is given below:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Example Where and How Keywords are Used in the Program

Example:

```
#include<stdio.h>
int main()
{
float a, b;
printf("Showing how keywords are used.");
return 0;
}
```

In the above program, **float** and **return** are keywords. The float is used to declare variables, and return is used to return an integer type value in this program.

C Constants

Constants are like a variable, except that their value never changes during execution once defined.

C Constants is the most fundamental and essential part of the C programming language. Constants in C are the fixed values that are used in a program, and its value remains the same during the entire execution of the program.

- Constants are also called literals.
- Constants can be any of the [data types](#).
- It is considered best practice to define constants using only **upper-case** names.

- Outline:

- [#_Constant Definition in C](#)
- [#_Constant Types in C](#)
- [#_Integer Constant](#)
- [#_Real constant](#)
- [#_Single Character Constants](#)
- [#_String Constants](#)
- [#_Backslash character constant](#)

Constant Definition in C

Syntax:

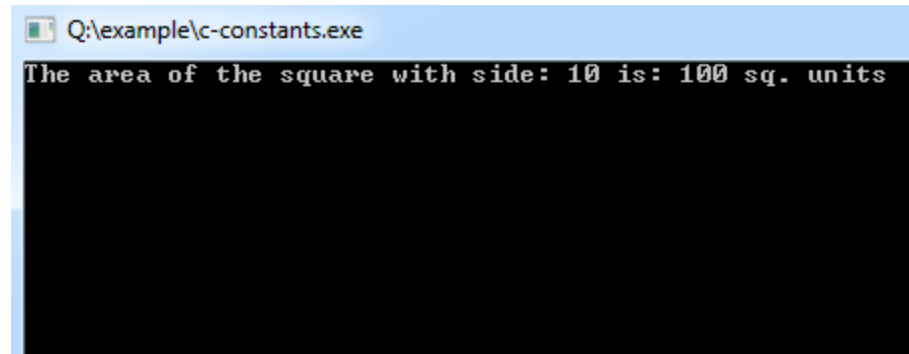
```
const type constant_name;
```

`const keyword` defines a constant in C.

Example:

```
#include<stdio.h>
main()
{
    const int SIDE = 10;
    int area;
    area = SIDE*SIDE;
    printf("The area of the square with side: %d is: %d sq. units"
        , SIDE, area);
}
```

Program Output:



Putting const either before or after the type is possible.

```
int const SIDE = 10;
```

or

```
const int SIDE = 10;
```

Constant Types in C

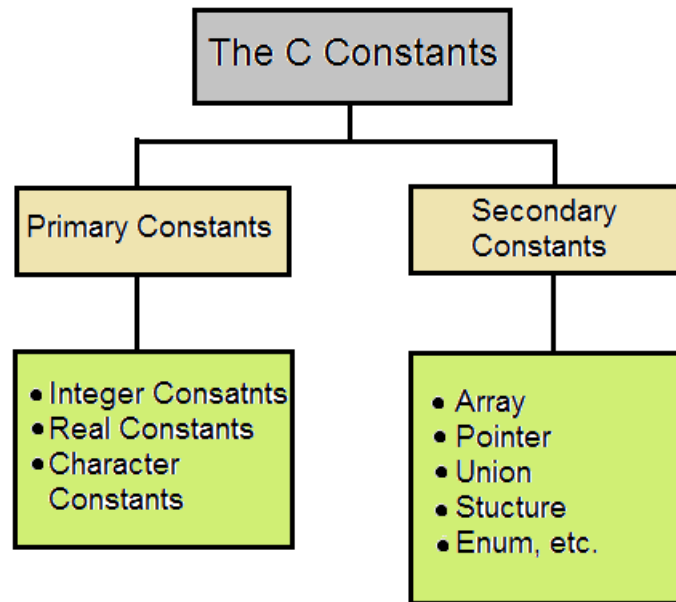
Constants are categorized into two basic types, and each of these types has its subtypes/categories. These are:
Primary Constants

1. Numeric Constants

- Integer Constants
- Real Constants

2. Character Constants

- Single Character Constants
- String Constants
- Backslash Character Constants



Integer Constant

It's referring to a sequence of digits. Integers are of three types viz:

1. Decimal Integer
2. Octal Integer
3. Hexadecimal Integer

Example:

15, -265, 0, 99818, +25, 045, 0X6

Real constant

The numbers containing fractional parts like 99.25 are called real or floating points constant.

Single Character Constants

It simply contains a single character enclosed within ' and ' (a pair of single quote). It is to be noted that the character '8' is not the same as 8. Character constants have a specific set of integer values known as ASCII values (American Standard Code for Information Interchange).

Example:

```
'X', '5', ','
```

String Constants

These are a sequence of characters enclosed in double quotes, and they may include letters, digits, special characters, and blank spaces. It is again to be noted that "G" and 'G' are different - because "G" represents a string as it is enclosed within a pair of double quotes whereas 'G' represents a single character.

Example:

```
"Hello!", "2015", "2+1"
```

Backslash character constant

C supports some character constants having a backslash in front of it. The lists of backslash characters have a specific meaning which is known to the compiler. They are also termed as "Escape Sequence".

For Example:

`\t` is used to give a tab

`\n` is used to give a new line

Constants	Meaning
\a	beep sound
\b	backspace
\f	form feed
\n	new line
\r	carriage return
\t	horizontal tab
\v	vertical tab
\'	single quote
\"	double quote
\\	backslash
\0	null

Secondary Constant

- [Array](#)
- [Pointer](#)
- [Structure](#)
- [Union](#)
- Enum